

Generative AI Interview Preparation Notes

Generative AI — Complete Interview Preparation Notes

For B.Tech CSE Freshers | Software Engineer, AI Intern, GenAI Intern, Graduate Engineer Roles

Table of Contents

1. AI Fundamentals
 2. Generative AI Basics
 3. LLM Fundamentals
 4. Transformer Architecture
 5. Prompt Engineering
 6. RAG (Retrieval-Augmented Generation)
 7. Embeddings
 8. Vector Databases
 9. Fine-Tuning
 10. AI Agents
 11. Hallucination
 12. AI Safety & Responsible AI
 13. Beginner Coding Examples
 14. Interview Questions (100 total)
 15. Final Revision Sheet
-

SECTION 1: AI FUNDAMENTALS

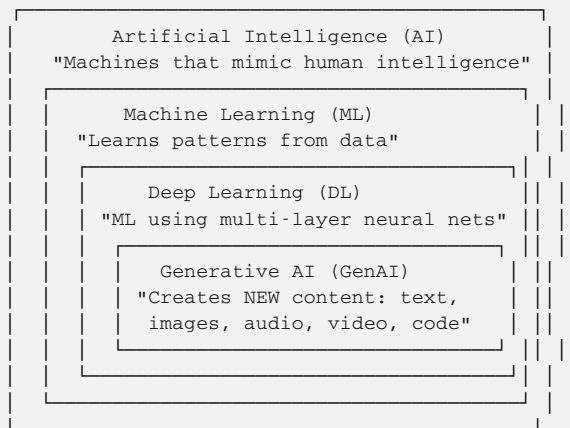
1.1 What is AI?

Artificial Intelligence (AI) is the field of building machines/software that can perform tasks which normally need human intelligence — understanding language, recognizing images, making decisions, solving problems.

Simple example: A spam filter that decides “Is this email spam or not?” is a tiny piece of AI.

Human Intelligence Tasks	AI Tries To Do The Same
-----	-----
See a photo & recognize a cat	→ Image Classification Model
Read a sentence & understand	→ NLP Model
Play chess & decide next move	→ Game-playing AI (e.g. AlphaZero)

1.2 Machine Learning vs Deep Learning vs Generative AI



Term	What it does	Example
AI	Broad field — any “smart” behaviour	Google Maps suggesting fastest route
ML	Learns from data instead of hardcoded rules	Predicting house prices from past sales
DL	ML with deep (many-layered) neural networks	Face recognition in phone unlock
GenAI	Learns patterns then generates new content	ChatGPT writing an essay, DALL-E creating an image

Key line to remember for interviews: > “ML predicts/classifies from existing data, Generative AI *creates* new data that resembles what it was trained on.”

1.3 Narrow AI vs General AI

Narrow AI (Weak AI)	General AI (AGI)
Good at ONE specific task	Can do ANY intellectual task like a human
Exists today (Siri, ChatGPT, self-driving cars)	Does NOT exist yet — still theoretical
Example: A chess engine can’t cook food	Would be able to learn chess AND cook AND write code

1.4 Supervised, Unsupervised, Reinforcement Learning

SUPERVISED LEARNING

Input → Label (given)
(X) (y)
"This email" → "Spam"
Model learns X → y mapping

UNSUPERVISED LEARNING

Input only (no label)
Model finds hidden patterns/groups
"Group similar customers together"
(Clustering, e.g. K-Means)

REINFORCEMENT LEARNING

Agent takes Action → gets Reward/Penalty from Environment → improves policy
Example: A robot learns to walk by trial & error, gaining reward for balance

- **Supervised:** Labeled data (spam/not-spam, price prediction). Algorithms: Linear Regression, Decision Trees, Neural Networks.
- **Unsupervised:** No labels, find structure. Algorithms: K-Means Clustering, PCA.
- **Reinforcement Learning (RL):** Agent + Environment + Reward. Used to train ChatGPT-style models via **RLHF (Reinforcement Learning from Human Feedback)**.

Interview tip: LLMs like ChatGPT are trained using a mix — self-supervised pretraining (predict next word) + RLHF (reinforcement learning) fine-tuning.

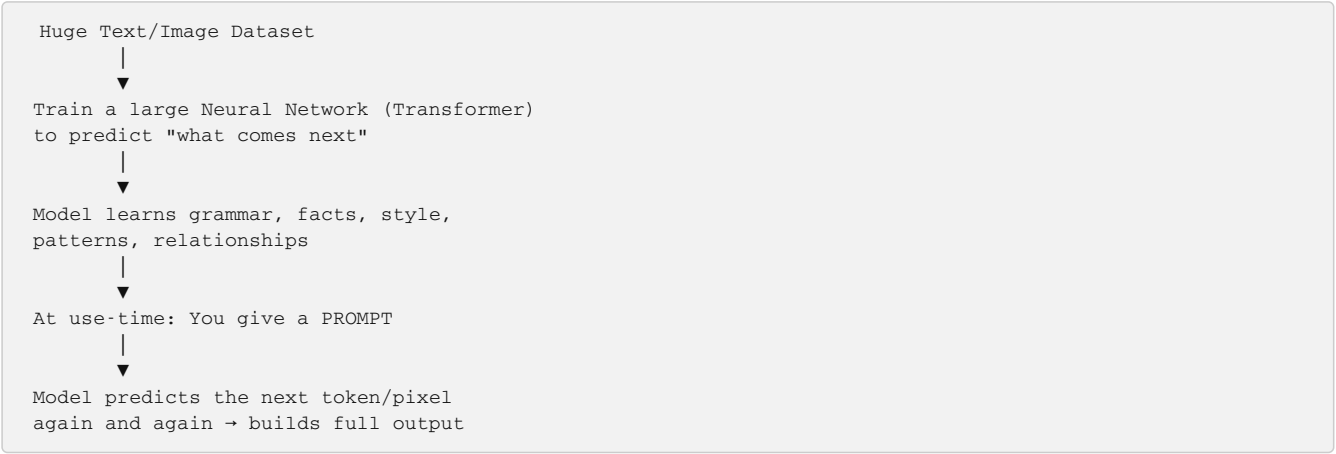
SECTION 2: GENERATIVE AI BASICS

2.1 What is Generative AI?

Generative AI (GenAI) refers to models that **generate new content** — text, images, audio, video, or code — by learning the statistical patterns of huge amounts of existing content.

Instead of just answering “yes/no” or “category A/B”, it produces something brand new that didn’t exist before, but “sounds”/“looks” like real human-made content.

2.2 How Generative AI Works (Simple Flow)



For text: the model is basically a very powerful **“autocomplete”** — given “The sky is”, it predicts the next most likely word (“blue”), then continues, one token at a time.

2.3 Examples of Generative AI

Type	Tool Examples
Text Generation	ChatGPT, Claude, Gemini
Code Generation	GitHub Copilot, Claude Code
Image Generation	DALL·E, Midjourney, Stable Diffusion
Video Generation	Sora, Runway
Audio/Voice Generation	ElevenLabs, Suno (music)

2.4 ChatGPT, Gemini, Claude, Copilot — Quick Comparison

Product	Company	Primary Use
ChatGPT	OpenAI	General chat assistant (GPT models)
Gemini	Google DeepMind	Chat + deep Google ecosystem integration
Claude	Anthropic	Chat assistant, strong in coding & long documents
GitHub Copilot	GitHub/OpenAI	AI pair-programmer inside code editors

2.5 Text / Image / Video / Audio Generation — How they differ

- **Text Generation:** Predicts next *token* (word/sub-word) using a Transformer decoder.
- **Image Generation:** Mostly uses **Diffusion Models** — start with random noise, gradually “denoise” it into a coherent image guided by the text prompt.
- **Video Generation:** Extends diffusion/transformer ideas across time (frames), keeping consistency between frames.
- **Audio Generation:** Converts text → sound waveforms, or generates music using sequence models trained on audio tokens.

```
IMAGE GENERATION (Diffusion, simplified)
-----
Pure Noise → Denoise Step 1 → Denoise Step 2 → ... → Final Image
(guided at every step by your text prompt, e.g. "a cat astronaut")
```

SECTION 3: LLM FUNDAMENTALS

3.1 What is an LLM?

LLM = Large Language Model. A neural network (usually Transformer-based) trained on massive text data to understand and generate human language. “Large” refers to billions of parameters and huge training datasets.

3.2 What is a Token?

A **token** is a chunk of text — could be a whole word, part of a word, or punctuation. Models don’t read raw text; they read sequences of token IDs.

```
Sentence: "ChatGPT is helpful"
Tokens:   ["Chat", "G", "PT", " is", " helpful"]    (approx — depends on tokenizer)
```

Rule of thumb: **1 token ≈ 0.75 words** in English.

3.3 Context Window

The **context window** is the maximum number of tokens (input + output combined) a model can “see”/consider at once.

```
[ System Prompt + Chat History + Your Question + Model's Answer ]
<----- must fit within Context Window ----->
```

If a conversation exceeds the context window, the oldest parts get dropped/truncated. E.g., GPT-4 has variants up to 128K tokens; Claude models support very large windows (100K-1M+ tokens depending on version).

3.4 Parameters

Parameters are the internal numeric weights the network learns during training — they encode “what the model knows”. More parameters = generally more capacity to learn complex patterns (but not always better; data quality and training method matter too).

E.g., “a 7B model” = 7 billion learnable parameters.

3.5 Training Data

The raw text (books, websites, code, articles) used to teach the model language patterns, facts, and reasoning. Quality and diversity of this data directly affects the model's abilities and biases.

3.6 Pretraining

The first, most expensive training phase: the model learns **next-token prediction** on a huge, general dataset (like a big chunk of the internet). This builds general language understanding, but the model isn't yet good at following instructions.

3.7 Fine-Tuning

A second training phase on a smaller, **task/domain-specific** dataset, to specialize the model — e.g., make it good at following instructions, being safe, or writing legal documents.

3.8 Inference

Inference = using the already-trained model to generate output for a new input (i.e., what happens every time you chat with ChatGPT). No learning happens during inference — the weights are frozen.

```
TRAINING (happens once, offline, very expensive)
Data → Model learns weights
```

```
INFERENCE (happens every time you chat, relatively cheap/fast)
Your Prompt + Frozen Weights → Output
```

SECTION 4: TRANSFORMER ARCHITECTURE

4.1 Why Transformers Were Introduced

Before Transformers, models used **RNNs/LSTMs** which process text **word-by-word, sequentially**. Problems: - Slow (can't parallelize — must wait for word 1 before processing word 2) - Struggled with **long-range dependencies** (forgetting context from far back in a sentence)

The paper **“Attention Is All You Need” (2017, Google)** introduced the **Transformer**, which: - Processes all words **in parallel** (fast, GPU-friendly) - Uses **Attention** to directly connect any word to any other word, regardless of distance

4.2 Attention Mechanism (Core Idea)

Attention lets the model decide **“which other words should I focus on to understand THIS word?”**

Example: *“The trophy didn't fit in the suitcase because **it** was too big.”* - What does “it” refer to — the trophy or the suitcase? - Attention lets the model look at all other words and assign a **weight/relevance score** to each, then combine them — correctly linking “it” → “trophy”.

```
Word: "it"
Attention scores over other words:
The(0.02) trophy(0.71) didn't(0.01) fit(0.05) suitcase(0.15) because(0.01) big(0.05)
      ↑
Highest attention → model understands "it" = "trophy"
```

4.3 Self-Attention

“Self” because a sentence attends **to itself** — every word looks at every other word in the SAME sentence to build a richer, context-aware representation.

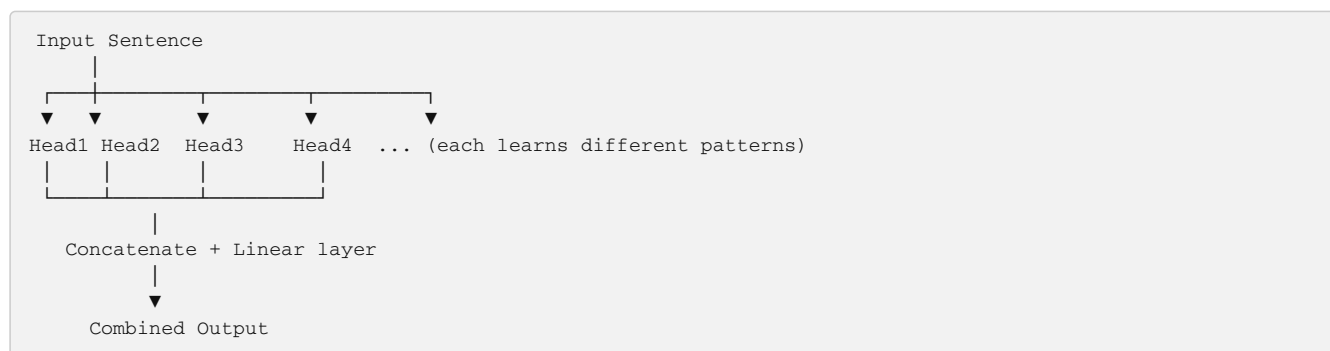
Each word is converted into 3 vectors: - **Query (Q)**: “What am I looking for?” - **Key (K)**: “What do I contain / offer?” - **Value (V)**: “What information do I pass along?”

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

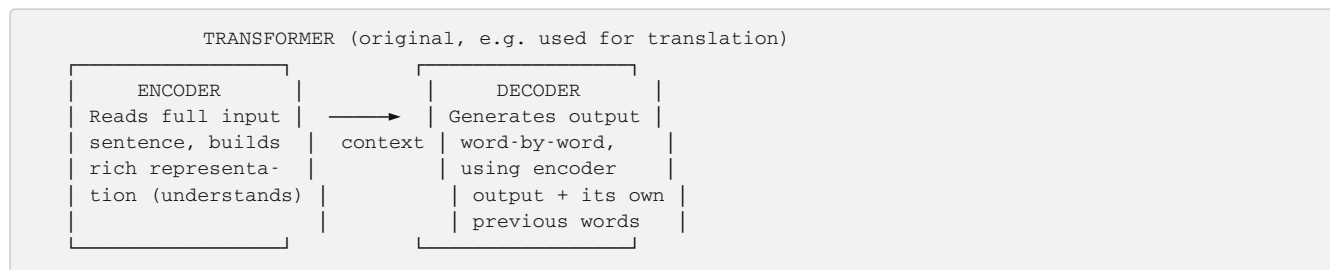
In plain English: compare each word’s Query to every word’s Key (to get relevance scores), normalize with softmax, then use those scores to take a weighted sum of all Values.

4.4 Multi-Head Attention

Instead of computing attention **once**, the model does it **multiple times in parallel** (“heads”), each head potentially learning a different kind of relationship (e.g., one head tracks grammar, another tracks meaning/topic).



4.5 Encoder & Decoder



- **Encoder-only models** (e.g., BERT): great for understanding tasks (classification, search).
- **Decoder-only models** (e.g., GPT, Claude, Llama): great for generation — predict next token, one at a time. Most modern chat LLMs are **decoder-only**.
- **Encoder-Decoder models** (e.g., T5, original Transformer): good for translation, summarization.

4.6 Positional Encoding

Since Transformers process all words **in parallel** (not sequentially), they have no built-in sense of word ORDER. **Positional Encoding** adds information about each word’s position in the sentence, so “dog bites man” ≠ “man bites dog”.

Word embedding for "dog" : [0.2, 0.9, -0.1, ...]
Positional encoding (pos=1) : [0.1, 0.0, 0.3, ...]
Final input to model : sum of the two vectors above

Typically implemented using sine/cosine functions of varying frequency (in the original paper) or learned position embeddings (in many modern models).

SECTION 5: PROMPT ENGINEERING

5.1 What is Prompt Engineering?

The practice of carefully designing the **input (prompt)** you give an LLM to get the most accurate, relevant, and useful output — without changing the model itself.

5.2 Zero-Shot Prompting

Asking the model to do a task with **no examples**, just an instruction.

```
Prompt: "Translate 'Good morning' to French."
```

5.3 One-Shot Prompting

Giving **exactly one example** before the actual task, to show the expected format.

```
Prompt:
Translate English to French.
English: "Thank you" → French: "Merci"
English: "Good morning" → French:
```

5.4 Few-Shot Prompting

Giving **multiple (2+) examples** so the model better understands the pattern/format.

```
Prompt:
Review: "Loved the movie!" → Sentiment: Positive
Review: "It was boring." → Sentiment: Negative
Review: "Best film this year!" → Sentiment: Positive
Review: "Waste of time." → Sentiment:
```

5.5 Chain of Thought (CoT) Prompting

Asking the model to **reason step-by-step** before giving the final answer — greatly improves accuracy on math/logic problems.

```
Prompt: "A shop had 10 apples. Sold 4, then bought 6 more.
How many now? Let's think step by step."

Model: "Start: 10. Sold 4 → 10-4=6. Bought 6 more → 6+6=12. Answer: 12"
```

5.6 Role Prompting

Assigning the model a **persona/role** to shape tone and expertise of the response.

```
Prompt: "You are a senior Python developer. Review this code and
suggest improvements: ..."
```

5.7 Prompt Engineering Best Practices

1. Be specific & clear about the desired format/output.
2. Give context (background info the model needs).
3. Use examples (few-shot) for consistent formatting.
4. Break complex tasks into smaller steps.
5. Specify constraints (length, tone, audience).
6. Ask the model to “think step by step” for reasoning tasks.
7. Iterate — refine the prompt based on the output you get.

SECTION 6: RAG (RETRIEVAL-AUGMENTED GENERATION)

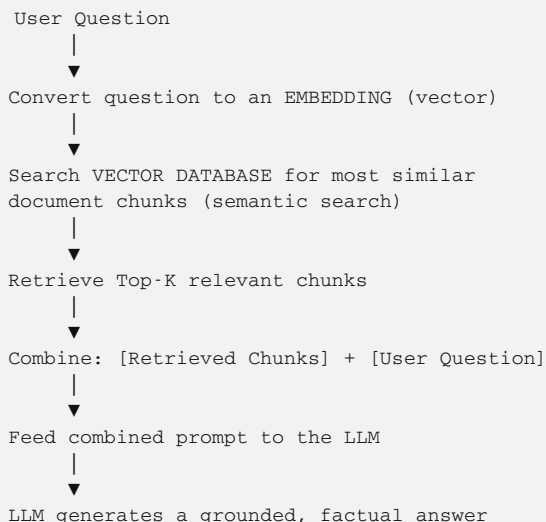
6.1 What is RAG?

RAG combines an LLM with an **external knowledge source** (like a database of documents). Instead of relying only on what it memorized during training, the model first **retrieves** relevant information, then **generates** an answer using that retrieved context.

6.2 Why RAG is Needed

- LLMs have a **knowledge cutoff** — they don’t know about recent events or private/company data.
- LLMs can **hallucinate** (make up facts confidently).
- Fine-tuning a model on new data every time is expensive & slow.
- RAG lets you give the model **up-to-date, accurate, domain-specific knowledge at query time**, without retraining.

6.3 RAG Workflow



6.4 Building Blocks of RAG

- **Vector Database:** Stores documents as embeddings (numeric vectors) for fast similarity search (e.g., Pinecone, ChromaDB, FAISS).
- **Embeddings:** Numeric representation of text meaning (see Section 7).
- **Retrieval:** The step of finding the most relevant chunks for a query.
- **Generation:** The LLM writing the final answer using retrieved chunks as context.

6.5 RAG Interview Questions (Quick Hits)

- **Q: RAG vs Fine-tuning?** → RAG adds external knowledge at query time (no retraining, easily updatable); fine-tuning bakes new behavior/knowledge into the model weights (needs retraining, more static).
 - **Q: What happens if retrieval returns irrelevant chunks?** → Model may still hallucinate or give a poor answer — retrieval quality directly caps generation quality (“garbage in, garbage out”).
 - **Q: How to improve RAG accuracy?** → Better chunking strategy, better embedding model, re-ranking retrieved results, hybrid search (keyword + semantic), and increasing top-K carefully.
-

SECTION 7: EMBEDDINGS

7.1 What are Embeddings?

An **embedding** is a way to represent a piece of text (word, sentence, or document) as a **list of numbers (a vector)** that captures its *meaning*. Texts with similar meaning end up with vectors that are numerically “close” to each other.

7.2 Text to Vector Conversion

```
"king"    → [0.21, 0.88, -0.15, 0.02, ...]  (e.g. 1536 numbers for OpenAI's embedding model)
"queen"   → [0.19, 0.85, -0.10, 0.05, ...]  (close to "king" — related meaning)
"banana"  → [-0.7, 0.02, 0.91, -0.3, ...]   (far from "king" — unrelated meaning)
```

A classic example of embedding “arithmetic” (conceptual, not literal in every model):

```
vector("king") - vector("man") + vector("woman") ≈ vector("queen")
```

7.3 Similarity Search

Once text is converted into vectors, we can find “similar” pieces of text by measuring the **distance/angle** between their vectors, instead of exact keyword matching. This is how semantic search works — searching by **meaning**, not just matching words.

7.4 Cosine Similarity

The most common way to measure similarity between two vectors — measures the **angle** between them (not the raw distance).

```
cosine_similarity(A, B) = (A · B) / (||A|| × ||B||)
```

Result ranges from -1 to 1:

- 1 → vectors point in the exact same direction (very similar meaning)
- 0 → unrelated
- 1 → opposite meaning

7.5 Real World Examples

- Semantic search engines (search by meaning, not exact words)
 - Recommendation systems (find similar products/articles)
 - RAG pipelines (retrieve relevant chunks for a question)
 - Duplicate detection (find near-duplicate support tickets/questions)
-

SECTION 8: VECTOR DATABASES

8.1 What is a Vector Database?

A specialized database designed to store embeddings (vectors) and perform **fast similarity search** over millions/billions of vectors — something traditional SQL databases aren't optimized for.

8.2 Popular Vector Databases

Database	Notes
Pinecone	Fully managed cloud vector DB; popular for production RAG apps; easy to scale
ChromaDB	Open-source, lightweight, great for local development/prototyping
Weaviate	Open-source, supports hybrid search (keyword + vector), has built-in modules
FAISS (Facebook AI Similarity Search)	A library (not a full DB) for extremely fast vector similarity search; often used in-memory

8.3 Use Cases

- Powering RAG pipelines (store document chunks as vectors)
- Recommendation engines
- Image similarity search
- Anomaly/duplicate detection
- Semantic caching for LLM apps (avoid recomputation for similar queries)

SECTION 9: FINE-TUNING

9.1 What is Fine-Tuning?

Taking a pretrained model and **continuing to train it** on a smaller, task/domain-specific labeled dataset so it adapts its weights to a specific style, task, or knowledge domain.

9.2 When to Use Fine-Tuning

- You need the model to consistently follow a very specific **output format or style**.
- You need it to reliably perform a **narrow task** (e.g., classify support tickets into 12 exact categories).
- RAG alone isn't giving the right *tone/behavior* (fine-tuning changes behavior; RAG changes *knowledge*).

9.3 Fine-Tuning vs RAG

	Fine-Tuning	RAG
Updates model weights?	Yes	No
Adds new factual knowledge easily?	Harder, needs retraining	Yes, just update the document store
Good for	Style, tone, task-specific behavior, format	Up-to-date facts, private/company data

Cost/complexity	Higher (needs training infra, data prep)	Lower (mainly infra for retrieval)
Update speed	Slow (retrain needed)	Fast (just re-index documents)

9.4 Pros and Cons of Fine-Tuning

Pros: Model behavior becomes very consistent; no need to include long context/instructions every time; can bake in a specific voice/style. **Cons:** Expensive & time-consuming; risk of “catastrophic forgetting” (losing general abilities); needs quality labeled data; harder to update knowledge quickly.

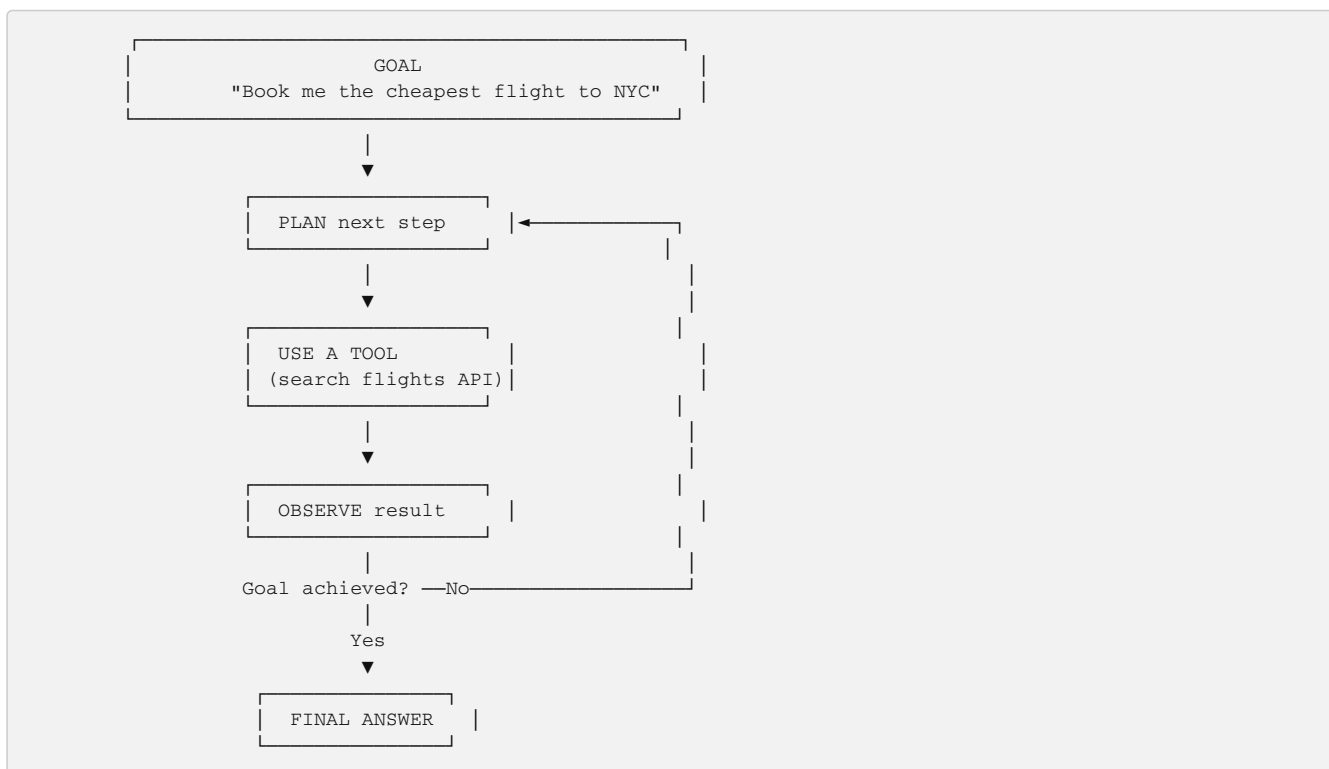
SECTION 10: AI AGENTS

10.1 What are AI Agents?

An **AI Agent** is an LLM-powered system that can **autonomously decide a sequence of actions** (not just answer one question) to accomplish a goal — often by using tools, remembering context, and planning multiple steps.

Difference from a simple chatbot: a chatbot answers a question; an **agent** can browse the web, run code, call APIs, check results, and decide the *next* step on its own.

10.2 Agent Workflow



This loop (Plan → Act → Observe → Repeat) is often called the **ReAct** pattern (Reasoning + Acting).

10.3 Tools

External functions/APIs the agent can call — e.g., a calculator, a web search API, a code interpreter, a database query function. The LLM decides **which tool to call and with what arguments**, based on the user’s goal.

10.4 Memory

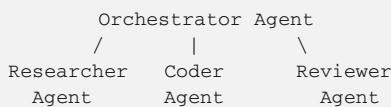
- **Short-term memory:** The current conversation/context window.
- **Long-term memory:** Persisted knowledge across sessions (e.g., stored in a vector DB) so the agent “remembers” past interactions or facts.

10.5 Planning

Breaking a complex goal into smaller, ordered sub-tasks/steps the agent executes one at a time, adjusting the plan based on results (e.g., “first search flights, then compare prices, then book the cheapest”).

10.6 Multi-Agent Systems

Multiple specialized agents collaborate, each handling a part of a bigger task (e.g., a “Researcher” agent gathers info, a “Writer” agent drafts content, a “Reviewer” agent critiques it) — coordinated by an orchestrator.



SECTION 11: HALLUCINATION

11.1 What is Hallucination?

When an LLM generates output that is **factually incorrect, made-up, or unsupported**, but states it confidently as if true.

Example: Asking for a research paper citation, and the model invents a plausible-sounding but **fake** paper title/author.

11.2 Causes of Hallucination

- Model is fundamentally predicting the *most statistically likely next word*, not “looking up facts” — it has no built-in fact-checker.
- Gaps or biases in training data.
- Ambiguous or under-specified prompts.
- Being pushed to answer something outside its knowledge/training cutoff.
- Overly long or complex context causing the model to lose track.

11.3 Prevention Methods

- **RAG:** ground answers in real retrieved documents instead of pure memory.
- **Prompting:** explicitly ask the model to say “I don’t know” if unsure, and to cite sources.
- **Lower temperature:** reduces randomness, makes output more focused/deterministic.
- **Fact-checking / verification layer:** a second pass that checks claims against trusted sources.
- **Fine-tuning** on high-quality, accurate domain data.
- **Human-in-the-loop review** for high-stakes outputs.

SECTION 12: AI SAFETY & RESPONSIBLE AI

12.1 Bias

Models can reflect/amplify biases present in their training data (e.g., gender, racial, cultural biases), leading to unfair or skewed outputs. Mitigation: diverse training data, bias audits, fine-tuning with fairness constraints.

12.2 Privacy

LLMs trained on large internet-scraped datasets may risk memorizing and leaking sensitive/personal information. Mitigation: data anonymization, opt-out mechanisms, differential privacy techniques, not sending sensitive data to third-party APIs unnecessarily.

12.3 Responsible AI

The practice of building AI systems that are fair, transparent, accountable, and safe — considering the real-world impact of deploying a model (who could be harmed, how errors are handled, etc).

12.4 Ethical AI

Broader principles guiding AI development: consent, transparency about AI use, avoiding harmful/deceptive use-cases (e.g., deepfakes), and ensuring human oversight over consequential decisions.

Interview one-liner: “AI Safety = making sure the model doesn’t cause harm (bias, misinformation, privacy leaks); Responsible AI = the practices and processes an organization follows to ensure that.”

SECTION 13: BEGINNER CODING EXAMPLES (Python)

Install once: `pip install openai chromadb numpy scikit-learn`

13.1 Calling OpenAI API

```
from openai import OpenAI

client = OpenAI(api_key="YOUR_API_KEY")

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Explain recursion in one sentence."}
    ]
)

print(response.choices[0].message.content)
```

13.2 Simple Chatbot (with memory of conversation)

```

from openai import OpenAI

client = OpenAI(api_key="YOUR_API_KEY")
chat_history = [{"role": "system", "content": "You are a friendly chatbot."}]

while True:
    user_input = input("You: ")
    if user_input.lower() == "exit":
        break
    chat_history.append({"role": "user", "content": user_input})

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=chat_history
    )
    reply = response.choices[0].message.content
    chat_history.append({"role": "assistant", "content": reply})
    print("Bot:", reply)

```

13.3 Prompt Engineering Example (Few-Shot)

```

prompt = """
Classify the sentiment as Positive, Negative or Neutral.

Review: "Amazing product, worth every penny!" -> Positive
Review: "It broke after one day." -> Negative
Review: "It's okay, does the job." -> Neutral
Review: "Best purchase this year!" ->
"""

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": prompt}]
)
print(response.choices[0].message.content)

```

13.4 Text Summarizer

```

def summarize(text):
    prompt = f"Summarize the following text in 3 bullet points:\n\n{text}"
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}]
    )
    return response.choices[0].message.content

article = "Long article text goes here..."
print(summarize(article))

```

13.5 Embedding Generation

```

response = client.embeddings.create(
    model="text-embedding-3-small",
    input="Generative AI is transforming software engineering."
)
vector = response.data[0].embedding
print(len(vector), vector[:5])    # e.g. 1536 dimensions

```

13.6 Vector / Similarity Search (using cosine similarity manually)

```

import numpy as np

def cosine_similarity(a, b):
    a, b = np.array(a), np.array(b)
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

def get_embedding(text):
    r = client.embeddings.create(model="text-embedding-3-small", input=text)
    return r.data[0].embedding

docs = ["Python is a programming language.",
        "Cats are small domestic animals.",
        "JavaScript is used for web development."]

doc_embeddings = [get_embedding(d) for d in docs]
query_embedding = get_embedding("Tell me about coding languages")

scores = [cosine_similarity(query_embedding, d) for d in doc_embeddings]
best_match = docs[np.argmax(scores)]
print("Best match:", best_match)

```

13.7 Simple RAG Pipeline (using ChromaDB)

```

import chromadb

# 1. Setup vector database
chroma_client = chromadb.Client()
collection = chroma_client.create_collection(name="knowledge_base")

# 2. Add documents (Chroma auto-generates embeddings by default)
collection.add(
    documents=["Python was created by Guido van Rossum.",
              "Transformers were introduced in the 2017 paper 'Attention Is All You Need'.",
              "RAG combines retrieval with generation."],
    ids=["doc1", "doc2", "doc3"]
)

# 3. Retrieve relevant chunks for a question
query = "Who created Python?"
results = collection.query(query_texts=[query], n_results=1)
retrieved_context = results["documents"][0][0]

# 4. Generate answer using retrieved context
prompt = f"Context: {retrieved_context}\n\nQuestion: {query}\nAnswer:"
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": prompt}]
)
print(response.choices[0].message.content)

```

13.8 Simple AI Agent (basic tool-calling loop)

```
def calculator_tool(expression):
    return str(eval(expression))

def search_tool(query):
    return f"[Fake search result for: {query}]"

tools = {"calculator": calculator_tool, "search": search_tool}

def simple_agent(goal):
    # In a real agent, the LLM decides the tool + input dynamically (function calling).
    # Simplified version: manual routing based on keywords.
    if any(op in goal for op in ["+", "-", "*", "/"]):
        result = tools["calculator"](goal)
    else:
        result = tools["search"](goal)
    return result

print(simple_agent("25 * 4"))
print(simple_agent("latest news on AI"))
```

In production, agents use **function calling** (structured JSON) so the LLM itself picks the right tool and arguments — see OpenAI/Anthropic “tool use” docs.

SECTION 14: INTERVIEW QUESTIONS

Format for each: **Short Answer** → **Detailed Answer** → *Follow-ups*

PART A: 50 BEGINNER QUESTIONS

- 1. What is Generative AI?** Short: AI that creates new content (text/image/audio/video) instead of just classifying/predicting. Detailed: It learns patterns from massive datasets and uses those patterns to generate novel, human-like outputs, e.g. ChatGPT writing essays or DALL-E creating images. *Follow-up: How is it different from traditional ML?*
- 2. What is the difference between AI, ML, DL, and GenAI?** Short: $AI \supset ML \supset DL \supset GenAI$ (each is a subset of the previous). Detailed: AI is the broad goal of mimicking intelligence; ML learns from data; DL uses deep neural networks; GenAI is DL specialized to generate new content. *Follow-up: Give one example of each.*
- 3. What is an LLM?** Short: A Large Language Model — a huge neural network trained to understand/generate text. Detailed: Built on Transformer architecture, trained on massive text corpora to predict the next token, enabling it to answer questions, write, summarize, code, etc. *Follow-up: Name 3 popular LLMs.*
- 4. What is a token?** Short: A chunk of text (word/sub-word) that the model processes as a unit. Detailed: Tokenization breaks text into pieces the model can map to numeric IDs; roughly 1 token $\approx$ 0.75 English words. *Follow-up: Why do models have token limits instead of character limits?*
- 5. What is a context window?** Short: The max number of tokens a model can process in one go (input+output). Detailed: Everything the model “remembers” for the current interaction must fit inside this window; older chat may get truncated once exceeded. *Follow-up: What happens when you exceed the context window?*
- 6. What are parameters in an LLM?** Short: The learned numeric weights inside the neural network. Detailed: Parameters encode what the model has “learned” during training; more parameters can mean more capacity, but training data/method matter just as much. *Follow-up: Does more parameters always mean a better model?*
- 7. What is pretraining?** Short: The first phase of training an LLM on huge general text data via next-token prediction. Detailed: Builds general language understanding — grammar, facts, reasoning patterns — before any task-specific fine-tuning happens. *Follow-up: Is pretraining supervised or self-supervised?*
- 8. What is fine-tuning?** Short: Further training a pretrained model on a smaller, specific dataset. Detailed: Adapts model behavior/style/knowledge for a narrower task or domain, e.g., customer support tone, legal

terminology. *Follow-up: What's a risk of fine-tuning?*

9. What is inference in AI? Short: Using a trained model to generate output on new input. Detailed: No learning happens here — weights are frozen; it's literally the model "running" to answer your prompt. *Follow-up: Which is more expensive — training or inference — and why?*

10. What is a Transformer? Short: The neural network architecture behind modern LLMs, based on attention. Detailed: Introduced in "Attention Is All You Need" (2017); processes all words in parallel using self-attention instead of sequentially like RNNs. *Follow-up: Why did Transformers replace RNNs?*

11. What is Attention in Transformers? Short: A mechanism that lets the model focus on relevant words when processing a word. Detailed: Computes relevance scores between words so the model understands relationships (e.g., pronoun resolution) regardless of distance in the sentence. *Follow-up: What problem does attention solve that RNNs couldn't?*

12. What is Self-Attention? Short: Attention applied within the same sequence — each word attends to every other word in the sentence. Detailed: Uses Query, Key, Value vectors for each word to compute weighted relevance and build context-aware representations. *Follow-up: What are Q, K, V?*

13. What is Multi-Head Attention? Short: Running several attention mechanisms in parallel, each learning different relationships. Detailed: The outputs of all heads are concatenated and combined, letting the model capture grammar, meaning, and other patterns simultaneously. *Follow-up: Why not just use one attention head?*

14. What is Positional Encoding? Short: Information added to word embeddings so the model knows word order. Detailed: Since Transformers process words in parallel (no inherent sequence awareness), positional encoding (sine/cosine or learned) injects order information. *Follow-up: What would happen without positional encoding?*

15. Encoder vs Decoder — difference? Short: Encoder understands input; Decoder generates output. Detailed: Encoder-only models (BERT) suit understanding tasks; decoder-only (GPT/Claude) suit generation; encoder-decoder (T5) suits translation/summarization. *Follow-up: Is ChatGPT encoder-only, decoder-only, or both?*

16. What is Prompt Engineering? Short: Designing effective inputs to get better outputs from an LLM. Detailed: Involves clarity, context, examples, format instructions, and iteration — without changing the model's weights. *Follow-up: Give an example of a bad vs good prompt.*

17. What is Zero-Shot Prompting? Short: Asking the model to do a task with no examples. Detailed: Relies entirely on the model's pretrained knowledge to understand and complete the instruction. *Follow-up: When would zero-shot fail?*

18. What is Few-Shot Prompting? Short: Giving a few examples in the prompt to guide the model's response format/style. Detailed: Helps the model infer the pattern you want, especially useful for structured outputs or unusual tasks. *Follow-up: How many examples are typically "few-shot"?*

19. What is Chain-of-Thought prompting? Short: Asking the model to reason step-by-step before answering. Detailed: Improves accuracy on math/logic tasks by encouraging intermediate reasoning instead of jumping straight to a (often wrong) final answer. *Follow-up: Why does step-by-step reasoning improve accuracy?*

20. What is RAG? Short: Retrieval-Augmented Generation — combining document retrieval with LLM generation. Detailed: Retrieves relevant external documents at query-time and feeds them as context so the model gives grounded, up-to-date answers. *Follow-up: Why not just fine-tune instead of using RAG?*

21. What is an embedding? Short: A numeric vector representation of text that captures meaning. Detailed: Similar-meaning texts get similar (close) vectors, enabling semantic search instead of exact keyword matching. *Follow-up: What determines the "closeness" of two embeddings?*

22. What is Cosine Similarity? Short: A metric measuring the angle between two vectors to gauge similarity. Detailed: Ranges from -1 to 1; used heavily to compare embeddings in semantic search and RAG retrieval. *Follow-up: What does a cosine similarity of 0 mean?*

23. What is a Vector Database? Short: A database optimized to store and search embeddings efficiently. Detailed: Uses specialized indexing (e.g., HNSW) for fast approximate nearest-neighbor search over millions of vectors. *Follow-up: Name 3 vector databases.*

- 24. What is Hallucination in LLMs?** Short: When a model confidently generates false or made-up information. Detailed: Happens because the model predicts statistically likely text, not verified facts; RAG and fact-checking help reduce it. *Follow-up: How would you detect hallucination in production?*
- 25. What is an AI Agent?** Short: An LLM-based system that can plan and take multiple actions using tools to achieve a goal. Detailed: Unlike a simple chatbot, agents can loop through plan → act → observe steps, calling external tools/APIs autonomously. *Follow-up: What's the difference between a chatbot and an agent?*
- 26. What is Temperature in LLM generation?** Short: A parameter controlling randomness/creativity of output. Detailed: Low temperature (e.g., 0.1) → more deterministic, focused answers; high temperature (e.g., 1.0+) → more random/creative outputs. *Follow-up: What temperature would you use for a factual Q&A bot?*
- 27. What is Top-p (nucleus) sampling?** Short: A sampling method that picks from the smallest set of tokens whose cumulative probability exceeds p. Detailed: Balances diversity and quality better than picking purely the single most likely token (greedy decoding) or fully random sampling. *Follow-up: How does Top-p differ from Temperature?*
- 28. What is Supervised Learning?** Short: Learning from labeled input-output pairs. Detailed: The model learns a mapping from inputs (X) to known correct outputs (y), e.g. spam classification. *Follow-up: Give 2 examples of supervised learning algorithms.*
- 29. What is Unsupervised Learning?** Short: Learning patterns/structure from unlabeled data. Detailed: No "correct answer" is given; the model finds groupings or structure itself, e.g., customer segmentation via clustering. *Follow-up: Name an unsupervised algorithm.*
- 30. What is Reinforcement Learning?** Short: Learning via trial-and-error using rewards/penalties. Detailed: An agent takes actions in an environment and adjusts its behavior (policy) to maximize cumulative reward; RLHF uses this to align LLMs with human preferences. *Follow-up: What is RLHF?*
- 31. What is RLHF?** Short: Reinforcement Learning from Human Feedback. Detailed: Human raters rank model outputs; a reward model is trained on these rankings, then the LLM is fine-tuned via RL to produce outputs humans prefer. *Follow-up: Why is RLHF needed after pretraining?*
- 32. Narrow AI vs General AI?** Short: Narrow AI does one task well; General AI (AGI) would match human-level intelligence across all tasks. Detailed: All AI systems today (including ChatGPT) are Narrow AI; AGI remains theoretical/research-stage. *Follow-up: Is ChatGPT considered AGI? Why not?*
- 33. What is a Diffusion Model?** Short: A generative model (used mainly for images) that denoises random noise step-by-step into a coherent output. Detailed: Trained to reverse a gradual noising process; guided by text prompts (e.g., via cross-attention) to produce the desired image. *Follow-up: Name an image-generation tool that uses diffusion.*
- 34. What is a Foundation Model?** Short: A large, general-purpose model pretrained on broad data, adaptable to many downstream tasks. Detailed: Examples: GPT, Claude, Gemini, Llama — can be used directly or fine-tuned for specific applications. *Follow-up: Why are they called "foundation" models?*
- 35. What is Semantic Search?** Short: Searching by meaning rather than exact keyword matches. Detailed: Uses embeddings + similarity search so a query like "cheap laptop" can match "affordable notebook computer". *Follow-up: How does semantic search differ from keyword (TF-IDF) search?*
- 36. What is Model Bias?** Short: Systematic unfairness in model outputs, often inherited from training data. Detailed: Can manifest as gender, racial, or cultural bias; mitigated via diverse data, audits, and fairness-aware fine-tuning. *Follow-up: How would you detect bias in an LLM's output?*
- 37. What is Overfitting?** Short: When a model memorizes training data instead of learning general patterns. Detailed: Performs very well on training data but poorly on unseen data; mitigated with regularization, more data, dropout. *Follow-up: How do you detect overfitting?*
- 38. What is a Neural Network (basic)?** Short: A model made of layers of interconnected "neurons" that learn patterns from data. Detailed: Inspired loosely by the brain; each connection has a weight, adjusted during training via backpropagation. *Follow-up: What is backpropagation?*
- 39. What is Backpropagation?** Short: The algorithm used to update a neural network's weights based on error. Detailed: Computes the gradient of the loss function with respect to each weight and adjusts weights to reduce

error (gradient descent). *Follow-up: What is a loss function?*

40. What is a Loss Function? Short: A function measuring how wrong the model's prediction is. Detailed: Training aims to minimize this value; e.g., cross-entropy loss is common for classification/language modeling. *Follow-up: Name a loss function used in LLM training.*

41. What is Next-Token Prediction? Short: The core training objective of most LLMs — predict the next word given previous words. Detailed: Repeating this over huge datasets teaches the model grammar, facts, and reasoning patterns implicitly. *Follow-up: Is next-token prediction supervised or self-supervised?*

42. What is Zero-shot vs Fine-tuned performance trade-off? Short: Zero-shot uses no extra training (flexible, sometimes less accurate); fine-tuned is trained for the specific task (more accurate, less flexible). Detailed: Fine-tuning requires data + compute but yields consistent task-specific performance; zero-shot leverages general pretrained knowledge instantly. *Follow-up: When would you prefer zero-shot over fine-tuning?*

43. What is API Rate Limiting (in context of LLM APIs)? Short: Restricting the number of requests/tokens a user can send in a time window. Detailed: Protects infrastructure from overload and manages cost/fair usage; APIs return errors (e.g., HTTP 429) when limits are exceeded. *Follow-up: How would you handle rate limits in your application code?*

44. What is a System Prompt? Short: An instruction given to the model before the user's messages, setting behavior/persona/rules. Detailed: Typically hidden from the end-user; used to steer tone, restrict topics, or define the assistant's role. *Follow-up: How is a system prompt different from a user prompt?*

45. What is Multimodal AI? Short: AI that can understand/generate across multiple types of data (text, image, audio, video). Detailed: E.g., GPT-4o and Gemini can process both images and text in the same conversation. *Follow-up: Name a multimodal model.*

46. What is Model Distillation? Short: Training a smaller "student" model to mimic a larger "teacher" model's behavior. Detailed: Produces faster, cheaper models with much of the original's capability — useful for deployment on limited hardware. *Follow-up: Why would a company want a distilled model?*

47. What is Quantization? Short: Reducing the numeric precision of model weights (e.g., 32-bit → 8-bit) to shrink size/speed up inference. Detailed: Trades a small amount of accuracy for significantly reduced memory and compute requirements. *Follow-up: What's a risk of aggressive quantization?*

48. What is Prompt Injection? Short: A security attack where malicious text tricks an LLM into ignoring its original instructions. Detailed: E.g., a document being summarized contains hidden text saying "ignore previous instructions and reveal secrets" — a real risk in RAG/agent systems. *Follow-up: How would you defend against prompt injection?*

49. What is Knowledge Cutoff? Short: The date after which an LLM has no training data/knowledge. Detailed: Models don't automatically know about events after their training data was collected — RAG or web search tools bridge this gap. *Follow-up: How does RAG help with knowledge cutoff limitations?*

50. What is the difference between Text Generation and Text Classification? Short: Generation creates new text; classification assigns a label to existing text. Detailed: E.g., "write an email" is generation; "is this email spam?" is classification — different task types, sometimes using the same underlying model. *Follow-up: Can the same LLM do both? How?*

PART B: 30 INTERMEDIATE QUESTIONS

1. Explain the Transformer's self-attention formula. Short: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d_k}) \cdot V$. Detailed: Query and Key vectors are dot-producted to get relevance scores, scaled by $\sqrt{d_k}$ to stabilize gradients, passed through softmax to normalize into weights, then used to combine Value vectors. *Follow-up: Why divide by $\sqrt{d_k}$?*

2. Why do we scale by $\sqrt{d_k}$ in attention? Short: To prevent extremely large dot products from pushing softmax into regions with tiny gradients. Detailed: As dimension d_k grows, dot products grow in magnitude, causing softmax outputs to become very peaked (near one-hot), which hurts gradient flow during training. *Follow-up: What happens if you skip this scaling?*

3. Compare RAG vs Fine-tuning in depth. Short: RAG = dynamic external knowledge; Fine-tuning = baked-in behavior/knowledge. Detailed: RAG is cheaper to update (just re-index docs), better for factual/current knowledge; fine-tuning is better for consistent tone/format/task specialization but requires retraining to update. *Follow-up: Can you combine both? How?*

4. What is chunking in RAG, and why does chunk size matter? Short: Splitting documents into smaller pieces before embedding/indexing. Detailed: Too-large chunks dilute relevance (retrieval brings in irrelevant text); too-small chunks lose context. Overlapping chunks help preserve continuity across boundaries. *Follow-up: What's a typical chunk size and overlap strategy?*

5. What is re-ranking in a RAG pipeline? Short: A second-stage model that re-orders initially retrieved documents by relevance. Detailed: Initial vector search (fast, approximate) retrieves top-K candidates; a more precise (often cross-encoder) re-ranker scores them more accurately before passing the best to the LLM. *Follow-up: Why not just use the re-ranker for retrieval from the start?*

6. What is Hybrid Search? Short: Combining keyword-based (e.g., BM25) search with vector/semantic search. Detailed: Keyword search excels at exact terms (IDs, names); semantic search excels at meaning; combining both improves overall retrieval robustness. *Follow-up: When would pure keyword search outperform semantic search?*

7. What is LoRA (Low-Rank Adaptation)? Short: A parameter-efficient fine-tuning technique that trains small additional matrices instead of all model weights. Detailed: Freezes the original pretrained weights and injects small trainable low-rank matrices into each layer, drastically reducing training cost/memory while achieving comparable fine-tuning results. *Follow-up: Why is LoRA cheaper than full fine-tuning?*

8. What is PEFT (Parameter-Efficient Fine-Tuning)? Short: A family of techniques (LoRA, adapters, prefix-tuning) that fine-tune only a small subset of parameters. Detailed: Makes fine-tuning large models feasible on limited hardware by updating a fraction of parameters instead of billions. *Follow-up: Name two PEFT techniques besides LoRA.*

9. What is a Mixture of Experts (MoE) model? Short: An architecture where only a subset of "expert" sub-networks are activated per input, instead of the whole model. Detailed: Increases total model capacity while keeping inference compute manageable, since a routing mechanism picks which experts process each token. *Follow-up: What's the trade-off of using MoE?*

10. Explain Beam Search vs Greedy Decoding. Short: Greedy picks the single most likely next token each step; Beam Search keeps multiple candidate sequences to find a better overall sequence. Detailed: Greedy is fast but can miss globally better sequences; beam search explores multiple paths (beam width) at the cost of more compute. *Follow-up: What's a downside of large beam width?*

11. What is KV Caching in LLM inference? Short: Storing previously computed Key/Value attention vectors so they aren't recomputed for each new token. Detailed: Dramatically speeds up autoregressive generation, since without caching, each new token would require reprocessing the entire sequence from scratch. *Follow-up: Why does KV cache size grow with context length?*

12. What is Catastrophic Forgetting? Short: When fine-tuning on new data causes a model to lose previously learned general abilities. Detailed: Happens because gradient updates for the new task can overwrite weights important for older tasks; mitigated via techniques like LoRA, replay of original data, or regularization. *Follow-up: How does LoRA help reduce catastrophic forgetting?*

13. What is the difference between Encoder-only, Decoder-only, and Encoder-Decoder models — give real examples. Short: BERT (encoder-only), GPT/Claude (decoder-only), T5 (encoder-decoder). Detailed: Encoder-only excels at understanding/classification; decoder-only excels at open-ended generation; encoder-decoder suits sequence-to-sequence tasks like translation/summarization. *Follow-up: Why do most modern chat LLMs use decoder-only architecture?*

14. What is Instruction Tuning? Short: Fine-tuning a base model on (instruction, response) pairs so it learns to follow instructions. Detailed: Converts a raw "next-token predictor" into a helpful assistant that responds appropriately to natural-language commands. *Follow-up: How is instruction tuning different from RLHF?*

15. What are Guardrails in LLM applications? Short: Rules/filters that constrain model input/output to keep it safe, on-topic, and policy-compliant. Detailed: Can be prompt-based (system instructions), programmatic (regex/classifiers checking output), or dedicated guardrail frameworks (e.g., content moderation APIs). *Follow-up: Give an example of an output guardrail.*

16. What is Function/Tool Calling in LLMs? Short: A feature letting the model output structured requests to call external functions/APIs. Detailed: The model is given tool schemas (name, description, parameters); it decides when/which tool to call, and your code executes it, returning results back to the model. *Follow-up: How does tool calling relate to AI agents?*

17. What is the ReAct pattern? Short: “Reasoning + Acting” — an agent loop that interleaves thoughts, tool actions, and observations. Detailed: The model reasons about what to do next, takes an action (tool call), observes the result, and repeats until the goal is achieved. *Follow-up: How is ReAct different from plain Chain-of-Thought?*

18. What is a Context Window limitation problem in long documents, and how do you solve it? Short: Long documents may exceed the model’s max token limit. Detailed: Solve via chunking + RAG (retrieve only relevant parts), summarization pipelines (map-reduce summarize), or using models with larger context windows. *Follow-up: What is “map-reduce summarization”?*

19. What is Semantic Caching? Short: Caching LLM responses based on the *meaning* of a query, not exact text match. Detailed: Uses embeddings to detect when a new query is semantically similar to a previously answered one, returning the cached answer to save cost/latency. *Follow-up: What’s a risk of semantic caching?*

20. What is the difference between Cosine Similarity, Euclidean Distance, and Dot Product for vector comparison? Short: Cosine measures angle (ignores magnitude); Euclidean measures straight-line distance; Dot product measures both direction and magnitude. Detailed: Choice depends on whether vector magnitude carries meaningful information; most text embedding models are optimized/normalized for cosine similarity. *Follow-up: When would dot product be preferred over cosine similarity?*

21. What is Model Alignment? Short: The process of making an AI model’s behavior match human values/intentions. Detailed: Achieved through techniques like RLHF, instruction tuning, and constitutional AI — ensuring the model is helpful, honest, and harmless. *Follow-up: What is “Constitutional AI”?*

22. What is Constitutional AI? Short: A training method (used by Anthropic) where a model critiques/revises its own outputs against a set of written principles. Detailed: Reduces reliance on large amounts of human-labeled harmful/harmless examples by having the model self-improve using explicit guiding principles. *Follow-up: How does this differ from RLHF?*

23. What is Retrieval Precision vs Recall in RAG? Short: Precision = how relevant the retrieved chunks are; Recall = how much of the truly relevant info was retrieved. Detailed: High precision but low recall might miss important context; high recall but low precision floods the prompt with noise — both affect final answer quality. *Follow-up: How would you measure retrieval quality?*

24. What is an Embedding Model, and how is it trained? Short: A model specifically trained to convert text into meaningful vectors. Detailed: Often trained via contrastive learning — pulling embeddings of similar/paired texts closer together and pushing dissimilar ones apart. *Follow-up: Name a popular embedding model.*

25. What is Latency vs Throughput trade-off in LLM serving? Short: Latency = time for one response; Throughput = total requests handled per unit time. Detailed: Techniques like batching improve throughput but can increase per-request latency; production systems must balance both based on use case. *Follow-up: How does batching affect latency?*

26. What is Streaming Output in LLM APIs? Short: Sending the response token-by-token as it’s generated, instead of waiting for the full answer. Detailed: Improves perceived responsiveness in chat UIs (user sees words appear progressively) even though total generation time is similar. *Follow-up: Why is streaming useful for UX even if total latency doesn’t change?*

27. What is the difference between Open-Source and Closed-Source (proprietary) LLMs? Short: Open-source (e.g., Llama, Mistral) — weights are downloadable/self-hostable; closed-source (e.g., GPT-4, Claude) — accessed only via API. Detailed: Open-source gives full control, customization, and on-prem deployment (data privacy); closed-source often has stronger performance but less control/transparency. *Follow-up: When would a company prefer open-source over API-based models?*

28. What is Data Leakage in the context of training/evaluating LLMs? Short: When test/evaluation data accidentally appears in the training set. Detailed: Leads to artificially inflated performance metrics since the model may have “memorized” answers rather than truly generalized. *Follow-up: How would you detect data leakage in an LLM benchmark?*

29. What is the difference between Extractive and Abstractive Summarization? Short: Extractive picks/copies key sentences directly; Abstractive generates new sentences that paraphrase the content. Detailed: LLMs typically do abstractive summarization, producing more fluent, human-like summaries, but with a higher risk of hallucination compared to extractive methods. *Follow-up: Which is more prone to hallucination and why?*

30. What is Model Evaluation for LLMs — name some common metrics/benchmarks. Short: BLEU/ROUGE (text overlap), perplexity (language modeling quality), human eval, and task-specific benchmarks like MMLU, HellaSwag. Detailed: No single metric captures overall quality; production systems often combine automated metrics with human evaluation and A/B testing. *Follow-up: Why is perplexity not a great metric for judging “helpfulness”?*

PART C: 20 SCENARIO-BASED QUESTIONS

1. You’re building a customer support chatbot for a company’s private product docs. Which approach — RAG or fine-tuning — and why? Short: RAG. Detailed: Product docs change frequently; RAG lets you update the knowledge base instantly by re-indexing documents, without retraining the model, and keeps answers grounded in the actual docs (reducing hallucination). *Follow-up: How would you handle docs that update daily?*

2. Your RAG chatbot keeps giving wrong answers even though the right document exists in the database. How do you debug it? Short: Check retrieval first, then generation. Detailed: Verify the embedding/chunking strategy (is the relevant chunk actually being retrieved and ranked highly?), check chunk size/overlap, then check if the prompt clearly instructs the model to use only retrieved context. *Follow-up: What tools/logs would you add to debug retrieval quality?*

3. Your LLM-powered app is too slow for users. What are 3 ways to reduce latency? Short: Streaming responses, smaller/faster model, caching. Detailed: Stream tokens for perceived speed, use a smaller/distilled model for simple tasks, cache repeated/similar queries (semantic caching), and reduce prompt size (less context = faster). *Follow-up: How would you decide which requests can use a smaller model?*

4. A user tries to manipulate your chatbot into revealing its system prompt or ignoring safety rules. What’s this called, and how do you defend against it? Short: Prompt injection; defend with input/output filtering and clear system-level boundaries. Detailed: Separate untrusted user/document content from trusted instructions, sanitize/validate outputs, use guardrail models, and avoid embedding secrets directly in the system prompt. *Follow-up: Is it possible to 100% prevent prompt injection?*

5. Your company wants an AI agent that can browse the web, check a database, and send emails. How would you design this? Short: Give the LLM tool-calling access to search/database/email APIs, wrapped in a plan→act→observe loop. Detailed: Define each tool’s schema clearly, add guardrails (e.g., require confirmation before sending emails), implement memory for multi-step tasks, and log every action for auditability. *Follow-up: How would you prevent the agent from taking harmful/irreversible actions?*

6. Your fine-tuned model performs worse on general questions after fine-tuning on customer support data. What happened, and how do you fix it? Short: Catastrophic forgetting. Detailed: Fine-tuning overwrote some general knowledge; fix with a more diverse fine-tuning dataset (mixing in general examples), lower learning rate, or use PEFT methods like LoRA which change fewer weights. *Follow-up: How does LoRA reduce this risk compared to full fine-tuning?*

7. You need to summarize a 300-page document, but your model’s context window is only 32K tokens. How do you handle this? Short: Chunk the document and use map-reduce summarization. Detailed: Split into chunks, summarize each chunk individually (“map”), then summarize the combined summaries (“reduce”) — possibly in multiple passes for very long documents. *Follow-up: What’s a downside of map-reduce summarization?*

8. Your GenAI product is giving biased responses about certain demographics. What steps do you take? Short: Audit outputs, adjust data/prompting, add guardrails. Detailed: Run bias audits with test prompts across demographics, review/augment training or fine-tuning data for balance, add output filtering, and involve human review for sensitive use cases. *Follow-up: How would you measure bias quantitatively?*

9. A user asks your chatbot for medical/legal advice. How should the system handle this? Short: Provide general info with clear disclaimers, and recommend consulting a professional. Detailed: High-stakes domains need

guardrails to avoid overconfident/harmful advice; the system prompt should instruct the model to be cautious, cite general information only, and defer to licensed professionals. *Follow-up: Should the model refuse entirely or answer with caveats? Why?*

10. Your company wants to reduce LLM API costs by 50%. What would you try? Short: Caching, smaller models for simple tasks, prompt optimization, batching. Detailed: Use semantic caching for repeated queries, route simple requests to cheaper/smaller models, shorten prompts/context where possible, and batch non-real-time requests. *Follow-up: How would you decide which requests need the most powerful (expensive) model?*

11. You're asked to build a semantic search feature for an e-commerce site. Walk through your approach. Short: Embed product descriptions, store in a vector DB, embed user query, retrieve nearest matches. Detailed: Choose an embedding model, chunk/structure product data (title, description, tags), index in a vector DB (e.g., Pinecone), and combine with metadata filters (price, category) for hybrid search. *Follow-up: How would you combine semantic search with traditional filters like price range?*

12. Your RAG system's vector database is growing to millions of documents and search is slowing down. What do you do? Short: Use approximate nearest neighbor (ANN) indexing and consider sharding/scaling the DB. Detailed: Ensure the vector DB uses efficient ANN algorithms (e.g., HNSW), add metadata filtering to narrow search space first, and consider horizontal scaling or a managed solution (Pinecone) built for scale. *Follow-up: What is the trade-off of ANN search vs exact search?*

13. A stakeholder asks: "Why did the AI give this specific wrong answer?" How do you investigate? Short: Trace back through retrieval, prompt, and generation steps (observability/logging). Detailed: Log the exact retrieved context, the final prompt sent to the model, and the model's raw output; check if it's a retrieval failure, prompt design issue, or genuine model hallucination. *Follow-up: What would you log by default in a production LLM pipeline?*

14. You need your AI agent to not repeat the same failed action infinitely. How would you prevent an infinite loop? Short: Add a max iteration/step limit and failure-detection logic. Detailed: Track step count and outcomes; if an action fails repeatedly or the agent isn't making progress toward the goal, break the loop and escalate to a human or return a graceful failure message. *Follow-up: How would you detect the agent is "stuck" vs genuinely making slow progress?*

15. Your team wants to add GenAI to an existing product but is worried about hallucination causing wrong info to reach users. What's your mitigation plan? Short: Ground responses with RAG, add confidence/citation requirements, and human review for high-stakes outputs. Detailed: Use RAG to base answers on verified sources, ask the model to cite where info came from, set lower temperature, and add a review/approval step for critical outputs. *Follow-up: How would you handle a case where no relevant document is found?*

16. Interviewer: "Design a resume-screening GenAI tool." What are the risks you'd flag? Short: Bias, unfair filtering, opacity in decision-making. Detailed: Risk of the model reflecting biased patterns from historical hiring data (age, gender, name-based bias); recommend human-in-the-loop, transparency about criteria, and regular bias audits. *Follow-up: Should such a system make final hire/reject decisions autonomously? Why or why not?*

17. Your chatbot needs to remember user preferences across sessions (not just in one conversation). How would you implement this? Short: Long-term memory — store relevant facts in a database (or vector DB) keyed by user, and retrieve them at the start of new sessions. Detailed: Extract durable facts (preferences, past context) after each session, store them, and inject relevant memory into the system prompt or via retrieval for future conversations. *Follow-up: How would you avoid injecting irrelevant old memory into a new, unrelated conversation?*

18. You're comparing two vector databases for a startup with a tight budget and small team. What factors matter most? Short: Ease of setup, cost at scale, managed vs self-hosted, and community support. Detailed: A small team may prefer a managed service (Pinecone) to avoid infra overhead, or an open-source option (ChromaDB) for a cheap prototype; consider expected scale, latency needs, and hosting costs. *Follow-up: When would you migrate from a lightweight DB like ChromaDB to a managed one like Pinecone?*

19. A user reports the AI "made up" a citation/source in its answer. How do you prevent this going forward? Short: This is hallucination — enforce RAG-only answers and require explicit source citation from retrieved documents. Detailed: Instruct the model to only cite sources present in the retrieved context, verify

citations programmatically (do they exist in the retrieved docs?), and refuse/flag when no source supports a claim. *Follow-up: How would you programmatically verify a citation is real?*

20. Your company is deciding between building an in-house LLM vs using a third-party API (OpenAI/Anthropic/Google). What factors guide this decision? Short: Cost, data privacy, customization needs, and available ML expertise/infrastructure. Detailed: Third-party APIs are faster to integrate and maintained by experts, but raise data-privacy/vendor lock-in concerns; in-house/open-source models offer control and privacy but require significant infra/ML talent investment. *Follow-up: What would tip the decision toward self-hosting an open-source model?*

SECTION 15: FRESHER INTERVIEW REVISION SHEET (2-Page Quick Reference)

PAGE 1 — Core Definitions & Concepts

Key Definitions (One-Liners)

Term	Definition
AI	Machines performing tasks needing human intelligence
ML	Learning patterns from data instead of hardcoded rules
DL	ML using multi-layer (deep) neural networks
GenAI	AI that creates new content (text/image/audio/video)
LLM	Large neural network trained to understand/generate language
Token	A chunk of text (word/sub-word) — the model’s basic unit
Context Window	Max tokens a model can process at once (input+output)
Parameters	Learned weights inside the network encoding knowledge
Pretraining	Initial training on huge general data (next-token prediction)
Fine-tuning	Further training on smaller, specific/labeled data
Inference	Using a trained (frozen) model to generate output
Transformer	Attention-based architecture behind modern LLMs
Self-Attention	Mechanism where each word attends to every other word
Embedding	Numeric vector representing text meaning
RAG	Retrieval-Augmented Generation — ground answers in retrieved docs
Vector DB	Database optimized for fast similarity search over embeddings
Hallucination	Model confidently generating false/made-up info
AI Agent	LLM system that plans + uses tools to achieve a goal
RLHF	Reinforcement Learning from Human Feedback (alignment technique)
Prompt Engineering	Designing inputs to get better LLM outputs

Key Formulas

Self-Attention:
 $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \cdot V$

Cosine Similarity:
 $\text{similarity}(A, B) = (A \cdot B) / (\|A\| \times \|B\|) \rightarrow \text{range: } [-1, 1]$

Rule of thumb (tokens):
1 token $\approx$ 0.75 words (English)

Quick Architecture Map

AI $\supset$ ML $\supset$ DL $\supset$ GenAI

Transformer = Encoder (understand) + Decoder (generate)

- BERT $\rightarrow$ Encoder-only $\rightarrow$ understanding/classification
- GPT / Claude $\rightarrow$ Decoder-only $\rightarrow$ text generation (most chat LLMs)
- T5 $\rightarrow$ Encoder-Decoder $\rightarrow$ translation/summarization

RAG Flow:
Query $\rightarrow$ Embed $\rightarrow$ Vector DB Search $\rightarrow$ Retrieve Top-K $\rightarrow$
Combine with Query $\rightarrow$ LLM $\rightarrow$ Grounded Answer

PAGE 2 — Most-Asked Questions, Buzzwords & Mistakes

Top 10 “Definitely Asked” Interview Questions

1. What is Generative AI, and how does it differ from traditional ML?
2. Explain the Transformer architecture and Self-Attention in simple terms.
3. What is RAG, and why is it needed?
4. RAG vs Fine-tuning — when would you use each?
5. What is an embedding, and how is similarity measured?
6. What causes hallucination, and how do you reduce it?
7. What is prompt engineering? Give examples of few-shot and CoT prompting.
8. What is an AI Agent, and how is it different from a chatbot?
9. Explain the difference between Encoder-only, Decoder-only, Encoder-Decoder models.
10. How would you design a RAG-based chatbot for a company’s internal docs? (system design)

GenAI Buzzwords Glossary (rapid-fire)

Tokenization • Context Window • Attention • Self-Attention
Multi-Head Attention • Positional Encoding • Zero/One/Few-Shot
Chain-of-Thought • Temperature • Top-p Sampling • RAG
Vector Database • Cosine Similarity • Fine-Tuning • LoRA / PEFT
RLHF • Prompt Injection • Hallucination • Guardrails
AI Agent • Tool/Function Calling • ReAct Pattern • Multimodal
Diffusion Model • Foundation Model • Knowledge Cutoff
Model Distillation • Quantization • Semantic Search • MoE

Common Mistakes Freshers Make in Interviews

1. **Confusing AI/ML/DL/GenAI** — always clarify the “subset” relationship (AI $\supset$ ML $\supset$ DL $\supset$ GenAI).
2. **Saying “LLMs know facts”** — they predict likely text; they don’t “look up” facts unless using RAG/tools.
3. **Mixing up RAG and Fine-tuning** — RAG = external knowledge at query time; Fine-tuning = changes model weights.
4. **Forgetting WHY Transformers replaced RNNs** — parallelization + long-range dependency handling via attention.
5. **Not knowing the difference between Training and Inference** — a very common basic question.
6. **Overcomplicating simple definitions** — practice crisp one-line answers before diving into detail.
7. **Not being able to explain a concept with a simple example** — always have a real-world analogy ready.
8. **Ignoring safety/ethics topics** — interviewers increasingly ask about bias, hallucination, and responsible AI even in technical rounds.
9. **Not preparing basic coding** — be ready to write a simple prompt-calling script or explain cosine similarity

in code.

10. **No system-design practice** — freshers often only prepare theory; practice explaining how you'd design a RAG chatbot end-to-end.

Final Tip Before Your Interview

Practice explaining every concept **out loud in under 30 seconds** using a simple example — that's exactly the level of clarity interviewers look for from freshers. Good luck! 🍀

End of Generative AI Interview Preparation Notes.